Addis Ababa University
አዲስ፡አበባ፡ዩኒቨርሲቲ

SEEK WISDOM, ELEVATE YOUR INTELLECT AND SERVE HUMANITY !

Information Storage and retrieval

Group members:

Abdulbasit Abdulsemed.....UGR/3594/17
Hiba Abdulhamid.....UGR/6520/17
Mahlet Tegete.....UGR/6422/17
Meron Getu.....UGR/0328/17
Nahom Aklil.....UGR/6192/17
Miftah Mohammed.....UGR/2405/17

Data & Indexing Lead

1. Introduction

The Data & Indexing task focuses on building a searchable document collection using **Elasticsearch**. The implementation is found in HW1/Create_Index.py, where raw documents from the **Cranfield dataset** are processed and indexed.

Main goals:

- Read raw documents from files
- Extract useful text content
- Store the documents in a searchable index

All later tasks (query processing, ranking, evaluation) depend on this indexed data.

2. Dataset Understanding

2.1 Dataset Used

The code clearly shows that the dataset comes from:

cran.all.1400.xml
cran.qry.xml
cranqrel.trec.txt

This is the **Cranfield collection**, a classic IR dataset containing **1,400 scientific abstracts**, **225 queries**, and **relevance judgments (qrels)**.

2.2 Structure of Dataset

Each document in cran.all.1400.xml is in **TREC XML format**:

```
<doc>
  <docno>67</docno>
  <title>dynamic stability of vehicles traversing ascending or descending paths through the atmosphere .</title>
  <author>tobak and allen.</author>
  <bib>naca tn.4275, 1958.</bib>
```

<text>dynamic stability of vehicles traversing ascending or descending paths through the atmosphere ...</text>
</doc>

- **docno** → unique ID
- **title** → document title
- **author** → author(s)
- **bib** → bibliography
- **text** → abstract content

Queries (cran.qry.xml) are stored as <top> elements with <num> and <title> fields. Relevance judgments (cranqrel.trec.txt) follow the TREC format: TOPIC ITERATION DOCNO RELEVANCY.

2.3 Key Observations

- We are **indexing individual documents**, not entire files.
- Each document has structured metadata (title, author, bib, text).
- Queries and qrels allow **retrieval evaluation** (precision, recall, MAP).

3. Document Parsing

3.1 Parsing Method

- **BeautifulSoup** → parse XML
- **Regex** → clean text

```
soup = BeautifulSoup(validPage, 'xml')
docs = soup.find_all('doc')
```

3.2 Text Extraction

Inside each document:

```
text = doc.find('text').get_text()
```

- Extracts the abstract (<text> field).
- Metadata (title, author, bib) can also be indexed separately.

3.3 Cleaning & Word Counting

```
for line in text.splitlines():
    word = re.sub('\s+', ' ', line).strip().split(' ')
    count += len(word)
```

- Removes extra spaces
- Splits into words
- Counts length (useful for ranking models like TF-IDF, BM25)

3.4 Final Document Structure

Indexed document example:

```
{
  "title": "...",
  "author": "...",
  "bib": "...",
  "text": "..."
}
```

4. Elasticsearch Indexing

- ***Connection:*** *es = Elasticsearch()*
- ***Indexing:***

```
es.index(
  index="cranfield",
  id=doc.docno.get_text().strip(),
  document=jsonDoc
)
```

- Index name: cranfield
- ID: docno
- Body: JSON with fields (title, author, bib, text)

5. Deliverables

Working Index → cranfield
Document Count → 1,400 documents
Structure Explanation → Each document has docno, title, author, bib, text

6. Observations

- Uses a **classic IR dataset** (Cranfield)
- Supports **retrieval evaluation** with queries + qrels
- Metadata indexing allows richer search (title, author, bib)
- Word count analysis can support ranking

7. Conclusion

The Data & Indexing phase successfully:

- Extracted documents from the Cranfield dataset
- Parsed and cleaned text + metadata
- Indexed documents into Elasticsearch

This provides a strong foundation for:

- Query processing
- Ranking models
- Evaluation with qr

Query Processing

Overview

Query processing is a critical component of the Information Retrieval system. Its primary role is to prepare and execute queries against the Elasticsearch index and retrieve the necessary statistical information required by the retrieval models. Unlike simple text preprocessing, this module acts as a bridge between the indexed document collection and the ranking algorithms.

1. Input Description

The input to the query processing module is a query file named *query_desc.51-100.short.txt*. Each query consists of a query number followed by a textual description.

Example:

51: Air traffic control

systems 52: International

airline safety In this format:

- The number (e.g., 51) represents the query identifier
- The text following the number represents the actual query

2. Query Preparation

1. Query Parsing

Each query is read from the input file and split into two components:

- Query ID
- Query text

The leading query number is removed from the text before further processing, as specified in the assignment instructions.

3.Text Normalization

The query text is normalized to ensure consistency with the indexed documents:

- Conversion to lowercase
- Removal of punctuation and non-alphanumeric characters

Example:

"Air Traffic Control Systems" → "air traffic control systems"

4. Tokenization

The cleaned query text is split into individual terms (tokens), which are used to retrieve matching documents.

Example:
"air traffic control systems" → ["air", "traffic", "control", "systems"]

5. Query Execution with Elasticsearch

After preprocessing, queries are executed against the Elasticsearch index. Instead of relying on Elasticsearch's built-in ranking mechanism, the system retrieves term-level statistics required for custom ranking.

For each query term, the following information is obtained:

- Term Frequency (TF) in each document
- Document Frequency (DF) across the collection
- Document length and average document length

These statistics are essential inputs for the retrieval models implemented separately.

6. Interaction with Retrieval Models

The processed queries and retrieved statistical data are passed to the ranking module (*Retrieval_Models.py*). Each retrieval model (TF-IDF, Okapi TF, BM25, and Language Models) uses this information to compute relevance scores for documents.

7. Output Role

The query processing module does not directly produce ranked results. Instead, it provides structured query data and term statistics, which are then used by the retrieval models to generate the final ranked output files.

8. Importance of Query Processing

This module ensures that queries are correctly interpreted and that accurate statistical information is retrieved from the index. Any errors in this stage would propagate to all retrieval models, negatively affecting the ranking results.

Ranked Results of TF-IDF, Okapi TF, and BM25 models

Introduction

Information Retrieval (IR) systems are designed to retrieve relevant documents from a large collection based on user queries. In this project, an IR system was implemented using the Cranfield dataset, a standard benchmark collection in IR research.
The system performs:

- Indexing of documents
- Query processing
- Document ranking using TF-IDF, Okapi TF, and BM25 models

The goal is to evaluate how effectively these models rank relevant documents and to compare their performance.

2. Dataset Description

The Cranfield dataset consists of:

- **cran.all.1400** → 1400 documents
- **cran.qry** → 225 queries
- **cranqrel** → relevance judgments

Each document contains:

- Title
- Abstract (used as the main content for indexing)

Each query represents a specific information need related to aerodynamics and fluid mechanics.

3. System Architecture

3.1 Indexing

- Documents were indexed and stored
- The main text field (body_text) was used
- Each document was uniquely identified by doc no

3.2 Query Processing

- Queries were selected and processed
- Preprocessing steps included:
    - Lowercasing
    - Removing extra spaces
    - Removing digits

3.3 Retrieval

- Queries were matched against all documents
- Term statistics (TF, DF, document length) were computed

3.4 Ranking

Documents were ranked using:

- TF-IDF
- Okapi TF
- BM25

4. Preprocessing

Both documents and queries underwent preprocessing:

- Tokenization
- Lowercasing
- Removal of numbers
- Basic normalization

This ensured consistency between queries and documents.

5. Retrieval Models

5.1 TF-IDF Model

TF-IDF measures the importance of a term in a document relative to the entire dataset.

$$tfidf(d,q) = \sum_{w \in q} okapi_t f(w,d) * log \frac{D}{df_w}$$

- D is the total number of documents in the corpus
- dfw is the number of documents which contain term w

5.2 Okapi TF Model

Okapi TF normalizes term frequency based on document length.

$$okapi\_tf(w,d) = \frac{tf_{w,d}}{tf_{w,d} + 0.5 + 1.5(\frac{len(d)}{avg(len(d))})}$$

- $tf_{w,d}$ is the term frequency of term w in document d
- $len(d)$ is the length of document d
- $avg(len(d))$ is the average document length for the entire corpus

5.3 BM25 Model

$$bm25(d,q) = \sum_{w \in q} \left[log\left(\frac{D+0.5}{df_w+0.5}\right) * \frac{tf_{w,d} + k_1 * tf_{w,d}}{tf_{w,d} + k_1\left((1-b) + b * \frac{len(d)}{avg(len(d))}\right)} * \frac{tf_{w,q} + k_2 * tf_{w,q}}{tf_{w,q} + k_2} \right]$$

- tf_{wq} is the term frequency of term w in query q
- K_1, k_2, and b are constants↘

⇄∪↕↕ ↘⇄⇨▽◀ ∪⇨◀⇨↑↓⇔→ ↕↕↞ ↓▽ ⇨ △⇨⇔↗↓⇔→ ⇨↙→↕△↓◀↑↘ ▶▽⇨⇦ ↓⇔ ↓⇔←↕△↘⇨◀↓↕⇔ △⇨◀△↓⇨◁⇨↙ ◀↕ ↘⇨⇨▽▶△⇨ ↑↕▷ △⇨↙⇨◁⇨⇔◀ ⇨ ⇦↕⇨▶↘⇨⇔◀ ↓▽ ◀↕ ⇨ ▼▶⇨△◢↘ ⇉◀ ↓▽ ⇨⇔ ↓↘▲△↕◁⇨⇦ ◁⇨△▽↓↕⇔ ↕← ^↹↖⇉⇆↹ ⇨⇔⇦ ↓▽ ▷↓⇦⇨↙◢ ▶▽⇨⇦ ↓⇔ ↘↕⇦⇨△⇔ ▽⇨⇨△⇨↑ ⇨⇔→↓⇔⇨▽

6. Implementation Details

- Programming Language: Python
- Libraries Used:
    - Elasticsearch
    - elasticsearch-dsl

- pickle

Steps:

1. Load and preprocess 1400 documents
2. Process selected queries
3. Compute TF, DF, and document lengths
4. Apply TF-IDF, Okapi TF, and BM25 formulas
5. Rank documents
6. Store results using pickle files

7. Results

Sample Queries Used

- Query 1: aerodynamics flow pressure
- Query 2: shock waves boundary layer
- Query 3: heat transfer fluid dynamics
- Query 4: turbulence flow velocity
- Query 5: airfoil lift drag

7.1 TF-IDF Results

Query 1

1. DocID=189 (16.8878)
2. DocID=174 (13.5324)
3. DocID=696 (12.8737)
4. DocID=423 (11.8074)
5. DocID=1382 (11.3998)
6. DocID=89 (11.1487)
7. DocID=310 (10.8977)
8. DocID=282 (10.6628)
9. DocID=1356 (10.0824)
10. DocID=216 (10.0170)

Query 2

1. DocID=329 (45.7169)
2. DocID=1313 (42.7935)
3. DocID=798 (39.8011)
4. DocID=132 (36.2215)
5. DocID=170 (33.7669)
6. DocID=1364 (30.4631)
7. DocID=335 (26.3245)
8. DocID=1244 (24.6311)
9. DocID=411 (23.8008)
10. DocID=569 (21.9718)

Query 3

1. DocID=962 (27.4430)
2. DocID=554 (21.4258)
3. DocID=623 (21.4258)
4. DocID=269 (20.2445)
5. DocID=872 (19.7868)
6. DocID=522 (19.6021)
7. DocID=571 (19.6021)
8. DocID=1328 (19.1523)
9. DocID=120 (17.9631)
10. DocID=101 (17.5054)

Query 4

1. DocID=99 (32.7476)
2. DocID=151 (28.7992)
3. DocID=976 (19.7420)
4. DocID=40 (15.7936)
5. DocID=459 (14.1316)
6. DocID=257 (13.6735)
7. DocID=80 (11.8452)
8. DocID=218 (11.8452)
9. DocID=977 (11.6917)
10. DocID=562 (11.4626)

Query 5

1. DocID=484 (38.5170)

2. DocID=70 (31.9139)
3. DocID=701 (30.3901)
4. DocID=1291 (24.5177)
5. DocID=39 (22.9175)
6. DocID=1329 (21.2999)
7. DocID=1239 (19.6375)
8. DocID=452 (18.0260)
9. DocID=1380 (17.9812)
10. DocID=699 (17.2557)

7.2 OKAPI TF results

Query 1:
1. DocID=216, Score=7.1402
2. DocID=634, Score=6.7365
3. DocID=225, Score=6.7138
4. DocID=689, Score=6.4599
5. DocID=753, Score=5.9996
6. DocID=11, Score=5.7230
7. DocID=244, Score=5.5396
8. DocID=792, Score=5.4280
9. DocID=902, Score=5.4280
10. DocID=1271, Score=5.2660

Query 2:
1. DocID=798, Score=14.1290
2. DocID=335, Score=12.9710
3. DocID=1364, Score=11.9364
4. DocID=345, Score=11.8725
5. DocID=569, Score=11.1893
6. DocID=328, Score=10.4974
7. DocID=132, Score=9.9350
8. DocID=1313, Score=9.9238
9. DocID=72, Score=9.7098
10. DocID=1157, Score=9.5141

Query 3:

1. DocID=269, Score=10.3872
2. DocID=872, Score=9.6007
3. DocID=120, Score=9.4094
4. DocID=110, Score=9.3409
5. DocID=873, Score=9.0917
6. DocID=260, Score=8.8597
7. DocID=267, Score=8.8597
8. DocID=398, Score=8.8154
9. DocID=1263, Score=8.8154
10. DocID=559, Score=8.3162

Query 4:

1. DocID=99, Score=10.9673
2. DocID=151, Score=10.7351
3. DocID=207, Score=8.0805
4. DocID=976, Score=7.5931
5. DocID=165, Score=7.4340
6. DocID=40, Score=7.1789
7. DocID=1278, Score=7.1181
8. DocID=1220, Score=6.7041
9. DocID=80, Score=6.5807
10. DocID=218, Score=6.5807

Query 5:

1. DocID=484, Score=11.6331
2. DocID=624, Score=10.8802
3. DocID=1380, Score=10.4476
4. DocID=1265, Score=9.8309
5. DocID=1329, Score=9.8176
6. DocID=1330, Score=9.5651
7. DocID=452, Score=9.4742
8. DocID=70, Score=9.4640
9. DocID=702, Score=9.1591
10. DocID=443, Score=8.9781

7.3 BM25 Result

Query 1:
1. DocID=634, Score=6.4877
2. DocID=11, Score=5.9955
3. DocID=753, Score=5.4019
4. DocID=284, Score=5.0910
5. DocID=1206, Score=4.6445
6. DocID=1, Score=4.5628
7. DocID=237, Score=4.4688
8. DocID=689, Score=4.2257
9. DocID=216, Score=4.1160
10. DocID=453, Score=3.7814

Query 2:
1. DocID=335, Score=12.7605
2. DocID=345, Score=10.9215
3. DocID=1364, Score=9.6866
4. DocID=411, Score=9.4503
5. DocID=517, Score=8.7199
6. DocID=1314, Score=8.5627
7. DocID=265, Score=8.4263
8. DocID=132, Score=8.3596
9. DocID=569, Score=8.3408
10. DocID=178, Score=8.2547

Query 3:
1. DocID=269, Score=10.4061
2. DocID=398, Score=10.3464
3. DocID=120, Score=9.7071
4. DocID=260, Score=9.1469
5. DocID=559, Score=9.0618
6. DocID=963, Score=9.0120
7. DocID=873, Score=8.4654
8. DocID=872, Score=8.3752
9. DocID=387, Score=8.0242
10. DocID=348, Score=7.7621

Query 4:
1. DocID=99, Score=9.3738
2. DocID=151, Score=9.0622
3. DocID=882, Score=8.3121
4. DocID=40, Score=7.2257
5. DocID=1331, Score=6.6601
6. DocID=207, Score=6.5665
7. DocID=397, Score=6.2047
8. DocID=976, Score=6.0659
9. DocID=1278, Score=5.8166
10. DocID=218, Score=5.7123

Query 5:
1. DocID=484, Score=10.3798
2. DocID=1329, Score=9.4902
3. DocID=624, Score=9.4481
4. DocID=1265, Score=9.2713
5. DocID=1256, Score=9.2650
6. DocID=702, Score=8.9261
7. DocID=70, Score=8.8906
8. DocID=1330, Score=8.7420
9. DocID=1124, Score=8.4542
10. DocID=279, Score=8.4283

8. Observations

- TF-IDF produced **higher score ranges** and emphasized rare terms
- Okapi TF provided **better normalization** for document length
- BM25 produced the **most balanced and realistic rankings**
- Rankings differed across models, confirming correct independent implementation

9. Challenges Faced

- Elasticsearch setup and security warnings
- Parsing Cranfield dataset correctly
- Computing DF and document lengths accurately

- Implementing formulas manually without built-in scoring

10. Conclusion

This project successfully implemented classical IR models on a real dataset. TF-IDF, Okapi TF, and BM25 were applied and compared. BM25 showed the best overall performance due to its balanced handling of term importance and document length.

Language Models

Introduction

In Information Retrieval (IR), ranking the documents based on their relevance to a query is crucial. Language Models (LM) are one of the approaches used to calculate the relevance score between a query and a document. They estimate the probability of observing the query terms in a given document, leveraging term frequency (TF), document frequency (DF), and collection frequency (CF).

This report will focus on two popular language models:
Laplace Smoothing (additive
smoothing) Jelinek-Mercer
Smoothing

Each model will be defined with the associated formulas, followed by a detailed example demonstrating their application.

1. Laplace Smoothing Model

1.1 Definition

Laplace smoothing is a probabilistic model that adjusts the probability estimation of unseen terms by adding a constant to the term frequency (TF). It ensures that the probability of every term is non-zero, even for terms that don’t appear in a given document. This is particularly useful for handling rare or unseen terms.

1.2 Formula

The formula for the Laplace smoothed probability of a word in a document is:

P(w|d) = TF(w, d) + 1/doc_length(d) + V

Where:
TF(w,d):is the term frequency of words in a document (how many times w appears in d). Doc_length:is the total number of words in a document .

V:is the vocabulary size (the total number of unique words in the entire corpus).

1.3 Explanation

- TF(w, d) + 1: The "+1" adds a smoothing effect to avoid a zero probability for words that do not appear in the document.
- doc_length(d) + V: The denominator normalizes the probability, adjusting for document length and ensuring consistency across all documents.

1.4 Example

Consider a small collection with only two documents:
Document 1 (D1): "air traffic control"
Document 2 (D2): "control systems
aviation"

Let's calculate the Laplace smoothed probability for the word "control" in Document 1. TF("control", D1) = 1 (control appears once in D1)

Document length (D1) = 3 (total words in D1)
Vocabulary size (V) = 5 (air, traffic, control, systems,
aviation) The Laplace smoothed probability for
"control" in D1 is:

P(w|d)=(1+1)/(3+5)=0.25

2. Jelinek-Mercer Smoothing Model

2.1 Definition

Jelinek-Mercer smoothing is another probabilistic model that combines the document-specific term probability with the global collection probability. The model assigns a weight to each of these components, controlled by a smoothing

parameter , which adjusts the balance between the document-specific and global probabilities.

2.2 Formula

The formula for the Jelinek-Mercer smoothed probability of a word in a document is:

P(w|d) = lambda*(TF(w, d)/doc_length(d)) + (1 - lambda)* (CF(w)/total_terms)

Where:
TF(w, d):is the term frequency of words in a document .
doc_length(d):is the length of the document (total number of words in d).
CF(w):is the collection frequency of word w , i.e., the total number of times it appears across all documents.

total_terms:is the total number of terms in the entire collection. lambda:is the smoothing parameter (typically set to 0.7).

2.3 Explanation

- Lambda: The parameter controls the balance between document-specific and global term probabilities. Higher values give more weight to document-specific probabilities, while lower values lean more on global collection statistics.
- Document Probability: calculates the probability of a term appearing in the document.
- Collection Probability: calculates the overall frequency of the term across all documents in the collection.

2.4 Example

Consider the same collection with two documents as above. Let’s calculate the Jelinek-Mercer smoothed probability for the word "control" in Document 1.

Assume:
Lambda=0.7
TF("control", D1) =
1

Document length (D1) = 3

CF("control") = 2 (control appears twice in the entire collection) Total terms = 6 (total number of terms in both documents)

The Jelinek-Mercer probability for "control" in D1 is:

P(control |D1) = 0.7*⅓+(1-0.7)*⅓=0.33..

3. Ranking Documents Using Language Models

3.1 Scoring Documents

After calculating the probabilities for each term in the query, the scores for all query terms are summed for each document. The final score for each document is a measure of how relevant it is to the query. The higher the score, the more relevant the document is to the query.

For a query with multiple terms, the final score for a document and query is:

S(d, Q) = sum(log(P(w|d)))

Where:
w E Q: are the terms in the query.
P(w|d):is the probability of observing terms in a document , calculated using either Laplace or Jelinek-Mercer smoothing.

3.2 Example

Let’s calculate the score for Document 1 for the query "air traffic control":
For Laplace smoothing:

S(D1, Q) = log(P(air|D1)) + log(P(traffic|D1)) +

log(P(control|D1)) For Jelinek-Mercer smoothing:

S(D1, Q) = log(P(air|D1)) + log(P(traffic|D1)) + log(P(control|D1))

The document with the highest score is deemed most relevant to the query.

Discussion

The results obtained from this project demonstrate the effectiveness of classical Information Retrieval (IR) models when applied to a structured dataset such as the Cranfield collection. The system successfully integrates document indexing, query processing, and multiple ranking models, allowing for a comprehensive comparison of retrieval performance.

One key observation is the variation in ranking behavior across the three models: TF-IDF, Okapi TF, and BM25. TF-IDF tends to assign higher scores to documents containing rare terms, which can sometimes lead to overemphasis on less contextually relevant documents. This is evident from the wider score ranges observed in the TF-IDF results, where documents with uncommon terms are ranked higher regardless of document length or term distribution.

In contrast, Okapi TF introduces normalization based on document length, which improves fairness in ranking longer documents. This model reduces the bias toward longer documents that naturally contain more terms, resulting in more balanced retrieval outputs. However, it still lacks a comprehensive mechanism for handling term saturation and query term importance.

BM25, as observed in the results, provides the most effective and balanced ranking among the three models. It incorporates both term frequency saturation and document length normalization, along with tunable parameters. This allows BM25 to better reflect real-world relevance, producing rankings that are more consistent and reliable across different queries. The consistency of top-ranked documents across multiple queries suggests that BM25 captures relevance more accurately than the other models.

Additionally, the implementation of Language Models (Laplace and Jelinek-Mercer smoothing) highlights an alternative probabilistic approach to document ranking. These models address the zero-probability problem and provide a more theoretically grounded framework for handling unseen terms. Jelinek-Mercer smoothing, in particular, offers flexibility by combining document-level and collection-level statistics, making it more adaptable compared to Laplace smoothing.

Another important aspect of the system is the role of preprocessing and query processing. Proper normalization, tokenization, and cleaning ensure consistency

between indexed documents and queries. Any errors at this stage would propagate through the ranking models and significantly degrade performance. The use of Elasticsearch for indexing and statistical retrieval further strengthens the system by providing efficient access to term-level data.

Despite the successful implementation, some limitations remain. The evaluation section lacks quantitative metrics such as Mean Average Precision (MAP) and Precision@K, which are essential for objectively comparing model performance. Additionally, parameter tuning for BM25 and smoothing parameters for language models was not explored, which could further improve retrieval effectiveness. The system also relies on basic preprocessing techniques and does not incorporate advanced methods such as stemming, stop-word removal, or semantic embeddings.

Conclusion

This project successfully developed a complete Information Retrieval system using the Cranfield dataset, covering all major stages: document indexing, query processing, ranking, and probabilistic modeling. The implementation demonstrates a clear understanding of classical IR techniques and their practical application using tools such as Elasticsearch and Python.

Among the evaluated models, BM25 proved to be the most effective ranking method due to its balanced handling of term frequency, document length, and relevance estimation. TF-IDF and Okapi TF also performed reasonably well but showed limitations in handling document normalization and term importance compared to BM25.

The inclusion of Language Models further enriches the system by introducing probabilistic ranking approaches that handle unseen terms more effectively. Jelinek-Mercer smoothing, in particular, offers a flexible and robust alternative to traditional models.

Overall, the project provides a strong foundation for understanding and implementing Information Retrieval systems. Future improvements could include incorporating standard evaluation metrics (such as MAP and Precision@10), optimizing model parameters, and integrating more advanced techniques like stemming, stop-word removal, and neural retrieval models. These enhancements would further improve the system’s accuracy and bring it closer to modern search engine performance.